Lab 2 Report

112062119 楊竺函

Proj03-01: Image Enhancement Using Intensity Transformations

1. Implementation

```python
img = cv2.imread(INPUT_IMAGE, cv2.IMREAD_UNCHANGED)
img = normalization(img)

output_log = IntTran.logTransform(img,c)
output_pow = IntTran.powerlawTransform(img,c,r)
print("Transformation Finished")

# print(output_log)
# print(output_pow)

normed_log = np.clip(output_log, 0.0, 1.0)
normed_pow = np.clip(output_pow,0.0,1.0)

print("Saving Ressult...")
out1 = cv2.imwrite("out_log.tif", normed_log)
out2 = cv2.imwrite("out_pow.tif", normed_pow)
```

```python
def normalization(img):
    img = img.astype(np.float32)
    maxval = img.max()
    minval = img.min()
    if maxval - minval == 0:
        return np.zeros_like(img, dtype=np.float32)
    return (img - minval) / (maxval-minval)
```

```python
def logTransform(input,c):
    return c * np.log(1+input)

def powerlawTransform(input, c, r):
    return c* np.power(input,r)
```

The input image is first read using cv2.imread(). It is then converted to the np.float32 data type and normalized so that all intensity values fall within the range [0,1]. The normalization is performed using the following equation:

value = (value - min) / (max - min).

After normalization, both the log transformation and the power-law transformation are applied to the source image with user-specified parameters.

The log transformation modifies the intensity values according to the following equation:

value = c * log(1+value).

On the other hand, the power-law transformation is done through another equation:

value = c * (value)^r.

After both transformations are completed, two new images are obtained. These images are adjusted again using **np.clip()** to ensure that their intensity values remain within the range [0,1], and the results are then written to output files.

2. Execution Result

original image

log transform (c = 0.5)

power-law transform
(c = 1, r = 2)

power-law transform
(c = 1, r = 1)

power-law transform
(c = 1, r = 0.5)

power-law transform
(c = 1, r = 0.25)

power-law transform
(c = 1, r = 0.1)

3. Discussion

(1) Difference between the original image, log transform, and power-law transform

The original image contains both very dark background regions and relatively bright structures. Due to this wide intensity distribution, some structures in the darker regions are less distinguishable in the original image.

After applying the log transformation, the darker intensity values are expanded while the brighter regions are compressed. As a result, details in darker areas of the image become more visible. The log transform slightly improves the visibility of structures around the darker background and soft tissues, while preventing the brightest regions from becoming overly dominant.

Compared with the log transform, the power-law transformation provides more flexible control of the image brightness. The power-law transform allows the intensity distribution to be adjusted depending on the selected gamma value. This makes it possible to selectively enhance darker regions or suppress bright regions.

(2) Difference between power-law results with different gamma values

When the gamma value is smaller than 1, the image becomes brighter and the darker regions are enhanced. In this case, structures within the spinal column and surrounding tissues become more visible. As the gamma value decreases further, the enhancement of dark regions becomes stronger. However, if it is too small, the image may become overly bright and some contrast between structures may be reduced.

When the gamma value is equal to 1, the transformation becomes linear. In theory, if the scaling constant $c = 1$, the resulting image should be identical to the original image. However, in practice, slight differences may still appear due to normalization.

On the other hand, when the gamma value is greater than 1, the overall brightness of the image decreases and darker regions become even darker.

Therefore, moderate gamma values smaller than 1 generally produce better visual enhancement for this image by improving the visibility of structures in darker regions.

Proj03-02: Histogram Equalization

1. Implementation

(1) main program

```
img = cv2.imread(INPUT_IMAGE, cv2.IMREAD_UNCHANGED)

output, T = histEqualization(img)

drawTransformation(T)

out_hist = imageHist(output)
drawHistogram(out_hist, "transformed")

cv2.imwrite("out.tif", output)
```

The main program first reads the input image and calls **histEqualization()** to perform histogram equalization on the original image. After the processed image and the transformation function are returned, the program performs the following three tasks. First, it calls **drawTransformation()** to plot the transformation function. Second, it calls **imageHist()** to compute the histogram of the transformed image. Third, it calls **drawHistogram()** to plot the histogram of the transformed image. Finally, the transformed image is generated and saved to the output file.

(2) imageHist()

```
def imageHist(input):
    histVector = np.zeros(256, dtype=np.float32)

    for row in input:
        for pix in row:
            histVector[pix] += 1

    return histVector
```

The function **imageHist()** is used to count the occurrence of each intensity level in a given image. It iterates through the entire image and accumulates the number of pixels for each intensity value.

(3) histEqualization()

```python
def histEqualization(input):

    # get histogram and draw histogram
    histVector = imageHist(input)
    drawHistogram(histVector, "original")

    # normalize to probability
    total_pixel = input.shape[0] * input.shape[1]
    histVector = histVector / total_pixel

    # compute transformation function
    S = np.zeros(256, dtype=np.float32)
    S[0] = 255 * histVector[0]

    for i in range(1,256):
        S[i] = S[i-1] + 255 * histVector[i]

    S = np.round(S).astype(np.uint8)
    print("Transformation Function is Computed")

    # generate transformed image
    transformedImage = S[input]

    return transformedImage,S
```

The function histEqualization() first computes and plots the histogram of the original image using **imageHist()** and **drawHistogram()**. Afterwards, it normalizes the histogram to obtain the probability distribution of each intensity level. Based on the normalized histogram, the transformation function is computed using the following equations: $\boldsymbol{S_0 = 255 \times histVector[0]}$, $\boldsymbol{S_k = S_{k-1} + 255 \times histVector[k]}$

The computed transformation function is then applied to the original image to generate the transformed image through intensity mapping.

(4) plotting functions

```python
def drawHistogram(h, name):

    plt.figure()
    plt.bar(np.arange(256), h)
    plt.title(f"Histogram of {name} image")
    plt.xlabel("Intensity Level")
    plt.ylabel("Count")
    plt.xlim([0, 255])
    plt.savefig(f"{name}_hist.png")
    plt.close()

    # print("sum prob:", np.sum(h))
    print(f"Histogram of {name} image is Plotted")
    return
```

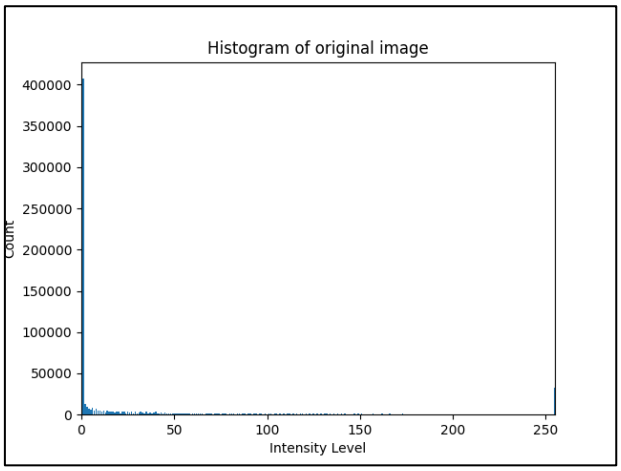
```python
def drawTransformation(T):
    x = np.arange(256)

    plt.figure()
    plt.step(x, T, where='mid')
    plt.title("Transformation Function")
    plt.xlabel("Input Intensity Level")
    plt.ylabel("Output Intensity Level")
    plt.xlim([0, 255])
    plt.ylim([0, 255])
    plt.savefig("transformation.png")
    plt.close()

    print("Transformation Function is Plotted")
    return
```

The function **drawHistogram()** is used to visualize the histogram of an image by plotting the distribution of pixel intensity values. It takes the histogram vector as input and generates a bar chart showing the count of each intensity level.

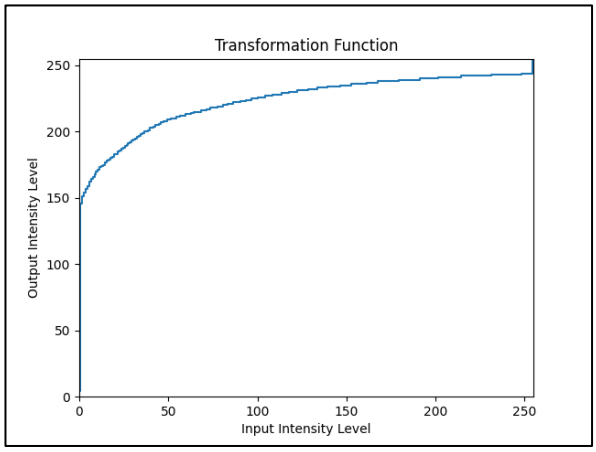
The function **drawTransformation()** plots the transformation function used in histogram equalization. It uses a step plot to illustrate how each input intensity level is mapped to a new output intensity level.

2. Execution Result

(a) original image

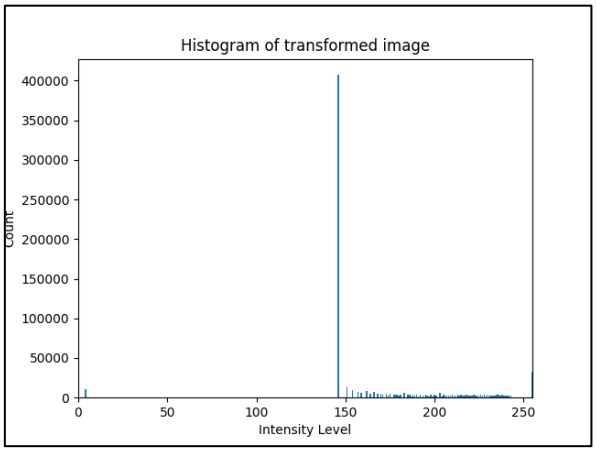


(b) histogram of (a)



(c) transformation function

(d) enhanced image



(e) histogram of (d)

3. Discussion

The histogram of the original image shows that a large number of pixels are concentrated at very low intensity levels, especially near intensity 0. This indicates that the image contains a large dark background and the dynamic range of gray levels is not fully utilized. As a result, the contrast of the image is limited and many details may be difficult to observe. After applying histogram equalization, the intensity values are redistributed according to the transformation function derived from the cumulative distribution of the original histogram. The transformation expands the low-intensity range and maps many dark pixels to higher intensity levels. Consequently, the histogram of the transformed image spreads over a wider range of gray levels. Although the resulting histogram is not perfectly uniform due to the discrete nature of pixel values, the overall contrast of the image is enhanced and the brightness distribution becomes more balanced. This demonstrates that histogram equalization effectively improves the visibility of image details by utilizing a broader range of intensity levels.

Proj03-03 Spatial Filtering

1. Implementation

(1) main program

```python
# prompt user to input mask
mask_raw = input(f"Please give {KERNEL_SIZE*KERNEL_SIZE} numbers for the mask: ").split()
mask = np.array(list(map(np.float32, mask_raw))).reshape(KERNEL_SIZE,KERNEL_SIZE)

# read the image
img = cv2.imread(INPUT_IMAGE,cv2.IMREAD_UNCHANGED)
img = normalization(img)

# padding zeros
pad = KERNEL_SIZE // 2
img = np.pad(img, pad, mode= "constant")

# applying spatial filtering
output = spatialFiltering(img,mask)
output = np.clip(output,0.0,1.0)

# write to output file
cv2.imwrite("out_spatialfilter.tif", output)
```

The main program can be divided into five steps. First, it prompts the user to input a mask for filtering and converts the raw input string into a 2-D array. Second, it reads the input image and performs normalization on the entire image using the normalization function implemented in Proj03-01. Next, the image is padded with zeros using **np.pad()**. After that, the program calls **spatialFiltering()** to perform convolution on the image using the user-specified mask. Finally, the output image is adjusted again using **np.clip()** to ensure that their intensity values remain within the range [0,1] and saved to an output file.

(2) spatialFiltering()

```python
def spatialFiltering(input, mask):

    a = KERNEL_SIZE // 2
    h,w = input.shape

    transformedimage = np.zeros((h - 2*a, w - 2*a), dtype=np.float32)

    for i in range(a,h-a):
        for j in range(a,w-a):
            conv_sum = 0

            for s in range(-a, a+1):
                for t in range(-a, a+1):
                    conv_sum += mask[s+a,t+a] * input[i-s, j-t]

            transformedimage[i-a,j-a] = np.float32(conv_sum)

    return transformedimage
```

The spatialFiltering() function performs 2-D convolution between the padded input image and the user-specified mask. It implements the following formula with some modified details: $g(x,y) = w(x,y) * f(x,y) = \sum_{s=-a}^{a} \sum_{t=-b}^{b} w(s,t) f(x-s, y-t)$

First, it calculates the padding size based on the kernel size and obtains the dimensions of the padded image. Then, it initializes an output image with the size of the original image. The function scans the padded image pixel by pixel. For each pixel location, it computes the convolution sum by multiplying the corresponding mask elements with the neighboring pixels in the input image and accumulating the results. Finally, the computed value is assigned to the corresponding location in the output image.

2. Execution Result

original image

filtered image

mask used:
[[0.11111,0.11111,0.11111],
[0.11111,0.11111,0.11111],
[0.11111,0.11111,0.11111]]

Proj03-04 Enhancement Using the Laplacian

1. Implementation

(1) main program

```python
# prompt the user to input scale and kernel size
scale = np.float32(input("Please input a scale for Laplacian Filter: "))
global KERNEL_SIZE
KERNEL_SIZE = int(input("Please specify the mask size (can be 3, 5, or 7): "))
SF.KERNEL_SIZE = KERNEL_SIZE # make sure SF.spatialFiltering can access correct value
useDiagonal = False
if KERNEL_SIZE == 3:
    useDiagonal = input("Including Diagonal Terms? (Y/N) ").strip().lower()

# preprocessing
img = cv2.imread(INPUT_IMAGE,cv2.IMREAD_UNCHANGED)
img = SF.normalization(img)

# select corresponding laplacian mask
laplacianMask = selectMask(useDiagonal)

# laplacian filtering
output, scaledLaplacian = laplacianFiltering(img, laplacianMask, scale)

# write to output file
output = np.clip(output,0.0,1.0)
cv2.imwrite("out_scaledlaplacian.tif", scaledLaplacian)
cv2.imwrite("out_laplacianfilter.tif", output)
```

The main program performs the following steps. First, it prompts the user to input the **scale value**, the **mask size**, and whether to include **diagonal terms** in the kernel if the mask size is **3×3**. Second, it reads the input image and preprocesses it using the normalization function implemented in Proj03-03. Third, the program selects the corresponding Laplacian mask according to the user input. Next, it calls the **laplacianFiltering()** function to perform Laplacian enhancement on the image. Finally, the enhanced image and the scaled Laplacian image are saved to output files.

(2) laplacianFiltering()

```python
def laplacianFiltering(input, laplacianMask, scale):

    # padding zeros
    pad = KERNEL_SIZE // 2
    paddedinput = np.pad(input, pad, mode="constant")

    # compute scaleLaplacian
    mask = scale * laplacianMask

    # call Spatial Filter
    scaledLaplacian = SF.spatialFiltering(paddedinput, mask)

    # enhancement
    transformedImage = scaledLaplacian + input

    return transformedImage, scaledLaplacian
```

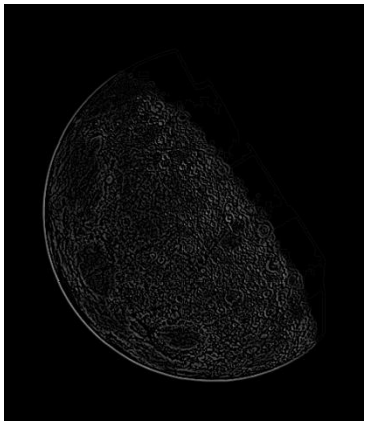
The laplacianFiltering() function performs image enhancement using the user-specified Laplacian mask and scale value. First, the function pads the input image with zeros according to the kernel size so that convolution can be applied to pixels near the image boundaries. Next, it multiplies the Laplacian mask by the scale value to obtain the scaled mask. Then, the function calls the previously implemented **spatialFiltering()**

function to compute the scaled Laplacian of the image. Finally, the scaled Laplacian image is added to the original image to generate the enhanced result. The function returns both the enhanced image and the scaled Laplacian image.

2. Execution Result

(1) Duplicate the result in Fig. 3.46

(a) input image



(b) Laplacian image

(c) sharpen image with c = -1

(d) sharpen image with c = -1 and kernel in Fig 3.45 (b)

(2) Sharpened image with different scale (fixed mask size = 3)

(a) scale = -1

(b) scale = -0.5

(c) scale = 1

(3) Sharpened image with different mask size (fixed c = -1)

(a) mask size = 3 * 3 (b) mask size = 5 * 5 (c) mask size = 7 * 7

3. Discussion

(1) Difference between different scale

The scale value mainly controls how strong the sharpening effect is. When the magnitude of the scale is small, such as -0.5, the enhancement effect is relatively mild and the image only becomes slightly sharper. When the scale is set to -1, the sharpening effect becomes more obvious and the edges in the image, such as the boundaries of craters on the moon, appear clearer.

If the scale is positive, the effect is quite different because the Laplacian component is added to the original image instead of being subtracted. As a result, the image may look less sharp or produce an opposite effect compared to the typical sharpening result. Therefore, the scale parameter has a direct impact on how strong the enhancement looks.

(2) Difference between different mask

Different mask sizes change how the Laplacian operator calculates the second derivative. A smaller mask, such as 3×3, only uses the nearest neighboring pixels, so it reacts more strongly to local intensity changes and produces a sharper result. Larger masks like 5×5 and 7×7 use a wider neighborhood, which makes the response slightly smoother. However, in this experiment the visual difference between different mask sizes is not very significant. The moon image already has very clear edges, so even the 3×3 mask is able to capture most of the important edge information. Larger masks produce slightly smoother results, but the overall sharpening effect looks quite similar.